

Database System Engineering and Distributed Backend Development

BANK DATABASE — Triggers & Stored Procedures

Name: Hansika

Roll No: 2520030237

Section: 7

Email: gunapatihansikareddy@klh.edu.in

Step 0 — Database & Table Setup (run once, before all questions)

```
CREATE DATABASE BankDB;
USE BankDB;

CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15),
    Email VARCHAR(100),
    City VARCHAR(50)
);

CREATE TABLE Account (
    Account_No INT PRIMARY KEY,
    Customer_ID INT,
    Account_Type VARCHAR(20),
    Balance DECIMAL(12,2) DEFAULT 0,
    Branch VARCHAR(50),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);

CREATE TABLE Bank_Transaction (
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
);

CREATE TABLE Loan (
    Loan_ID INT PRIMARY KEY,
    Customer_ID INT,
    Loan_Type VARCHAR(30),
```

```

        Loan_Amount DECIMAL(12,2),
        Interest_Rate DECIMAL(5,2),
        FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
    );

CREATE TABLE Transaction_Audit (
    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
    Transaction_ID INT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALUES
(101, 'Hansika', '9876543210', 'gunapatihansikareddy@klh.edu.in', 'Hyderabad'),
(102, 'Priya Sharma', '9876543211', 'priya@gmail.com', 'Vijayawada'),
(103, 'Arjun Reddy', '9876543212', 'arjun@gmail.com', 'Bangalore'),
(104, 'Sneha Rao', '9876543213', 'sneha@gmail.com', 'Chennai'),
(105, 'Kiran Kumar', '9876543214', 'kiran@gmail.com', 'Hyderabad');

INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES
(10001, 101, 'Savings', 50000, 'Hyderabad'),
(10002, 102, 'Savings', 75000, 'Vijayawada'),
(10003, 103, 'Current', 120000, 'Bangalore'),
(10004, 104, 'Savings', 45000, 'Chennai'),
(10005, 105, 'Current', 90000, 'Hyderabad');

INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES
(10001, 'DEPOSIT', 10000),
(10002, 'DEPOSIT', 15000),
(10003, 'WITHDRAW', 20000),
(10004, 'DEPOSIT', 5000),
(10005, 'WITHDRAW', 10000);

INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES
(501, 101, 'Home Loan', 5000000.00, 7.5),
(502, 102, 'Education Loan', 1000000.00, 6.5),
(503, 103, 'Car Loan', 800000.00, 8.2),
(504, 104, 'Personal Loan', 500000.00, 10.5);

```

Output:

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
501	101	Home Loan	5000000.00	7.50
502	102	Education Loan	1000000.00	6.50
503	103	Car Loan	800000.00	8.20
504	104	Personal Loan	500000.00	10.50
NULL	NULL	NULL	NULL	NULL

Customer 1 Account 2 Bank_Transaction 3 Loan 4 Apply Revert

Question 1 — Display all accounts in a particular branch

Query:

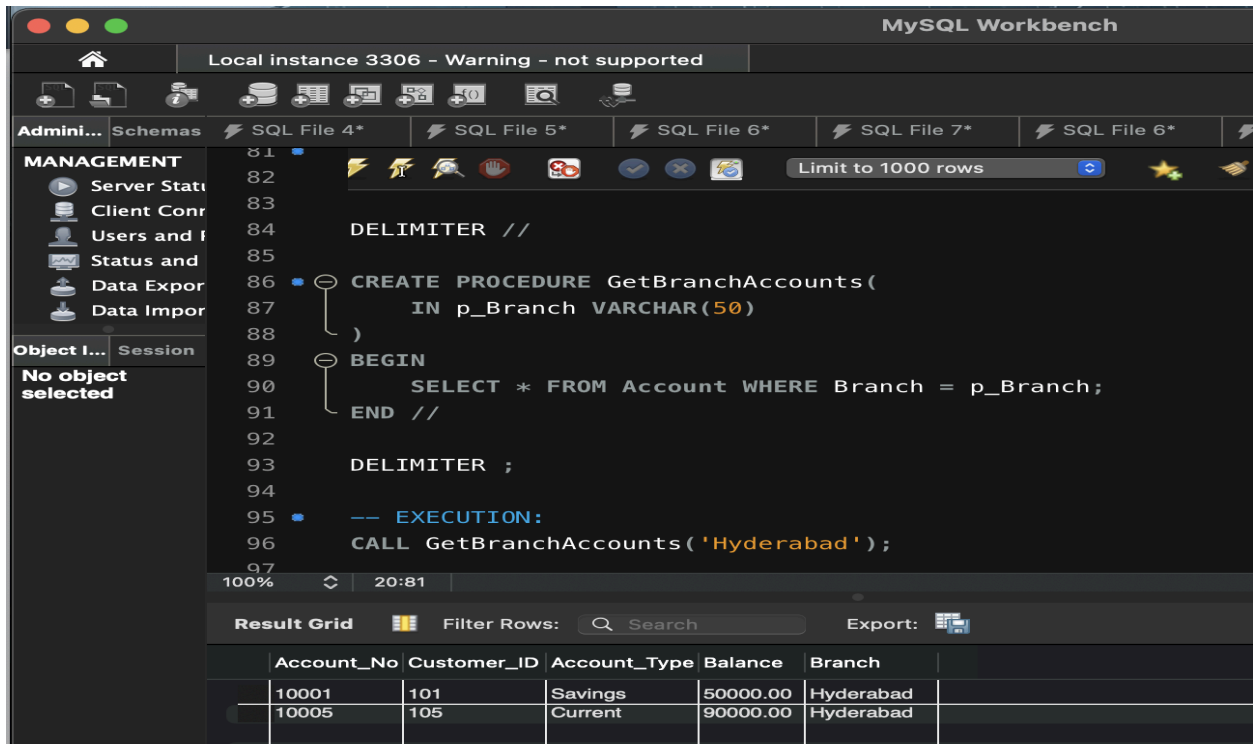
```
DELIMITER //
```

```
CREATE PROCEDURE GetBranchAccounts(  
    IN p_Branch VARCHAR(50)  
)  
BEGIN  
    SELECT * FROM Account WHERE Branch = p_Branch;  
END //
```

```
DELIMITER ;
```

```
-- EXECUTION:  
CALL GetBranchAccounts('Hyderabad');
```

Output:



The screenshot shows the MySQL Workbench interface. The left sidebar has a 'MANAGEMENT' tab with options like 'Server Status', 'Client Connections', 'Users and Privileges', 'Status and Performance', 'Data Export', and 'Data Import'. Below this is an 'Object Inspector' showing 'No object selected'. The main editor displays the SQL code from the previous block. The bottom panel shows the 'Result Grid' with the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10005	105	Current	90000.00	Hyderabad

Question 2 — Total balance of all accounts of a customer (OUT parameter)

Query:

```
DELIMITER //
```

```
CREATE PROCEDURE GetTotalBalance(  
    IN p_Customer_ID INT,  
    OUT p_TotalBalance DECIMAL(12,2)  
)
```

```

BEGIN
    SELECT SUM(Balance) INTO p_TotalBalance
    FROM Account
    WHERE Customer_ID = p_Customer_ID;
END //

DELIMITER ;

-- EXECUTION:
CALL GetTotalBalance(101, @Total);
SELECT @Total AS Total_Balance;

```

Output:

100%	14:116
Result Grid	Filter Rows: Search Export:
Total_...	
50000...	

Question 3 — Trigger — prevent account balance from going negative

Query:

```

DELIMITER //

CREATE TRIGGER PreventNegativeBalance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Balance cannot go negative';
    END IF;
END //

DELIMITER ;

-- TEST (should fail):
UPDATE Account
SET Balance = Balance - 999999
WHERE Account_No = 10001;

```

Output:

26	13:56:38	-- TEST (shoul... Error Code: 1644. Balance cannot go negative	0
----	----------	--	---

Question 4 — Trigger — auto-record every transaction into Transaction_Audit

Query:

```
DELIMITER //
```

```
CREATE TRIGGER AutoAuditTransaction
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    INSERT INTO Transaction_Audit
        (Transaction_ID, Account_No, Transaction_Type, Amount)
    VALUES
        (NEW.Transaction_ID, NEW.Account_No, NEW.Transaction_Type, NEW.Amount);
END //
```

```
DELIMITER ;
```

```
-- TEST:
INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount)
VALUES (10002, 'DEPOSIT', 3000);
```

```
-- CHECK:
SELECT * FROM Transaction_Audit;
```

Output:

The screenshot shows the SQL Developer interface with a script editor and an action output pane. The script being executed is as follows:

```
DELIMITER //
```

```
CREATE TRIGGER AutoAuditTransaction
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    INSERT INTO Transaction_Audit
        (Transaction_ID, Account_No, Transaction_Type, Amount)
    VALUES
        (NEW.Transaction_ID, NEW.Account_No, NEW.Transaction_Type, NEW.Amount);
END //
```

```
DELIMITER ;
```

```
-- TEST:
INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount)
VALUES (10002, 'DEPOSIT', 3000);
```

```
-- CHECK:
SELECT * FROM Transaction_Audit;
```

The action output pane shows the following error:

Time	Action	Response	Duration / Fetch Time
14:05:18	CREATE TRIGGER AutoAuditTransaction AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGI...	Error Code: 1046. No database selected Select the de...	0.0023 sec

Question 5 — Trigger — auto-increase balance on DEPOSIT

Query:

```
DELIMITER //
```

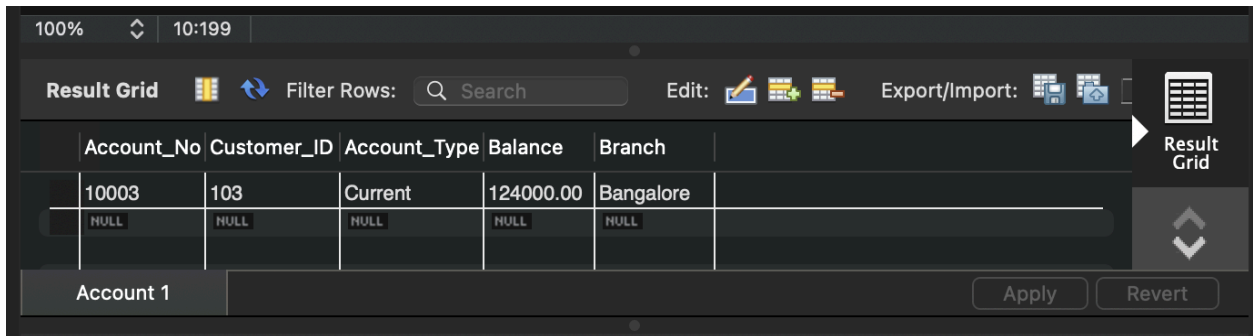
```
CREATE TRIGGER AutoIncreaseBalance
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Transaction_Type = 'DEPOSIT' THEN
        UPDATE Account
        SET Balance = Balance + NEW.Amount
        WHERE Account_No = NEW.Account_No;
    END IF;
END //
```

```
DELIMITER ;
```

```
-- TEST:
INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount)
VALUES (10003, 'DEPOSIT', 4000);
```

```
-- CHECK:
SELECT * FROM Account WHERE Account_No = 10003;
```

Output:



The screenshot shows a database application interface. At the top, there's a status bar with '100%' zoom and '10:199' time. Below it is a toolbar with 'Result Grid', 'Filter Rows', 'Search', 'Edit', and 'Export/Import' buttons. The main area displays a table with the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10003	103	Current	124000.00	Bangalore
NULL	NULL	NULL	NULL	NULL

Below the table, there's a section labeled 'Account 1' with 'Apply' and 'Revert' buttons. On the right side, there's a 'Result Grid' icon and a double arrow icon.

Question 6 — Trigger — auto-decrease balance on WITHDRAW

Query:

```
DELIMITER //
```

```
CREATE TRIGGER AutoDecreaseBalance
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Transaction_Type = 'WITHDRAW' THEN
        UPDATE Account
        SET Balance = Balance - NEW.Amount
    END IF;
END //
```

```

        WHERE Account_No = NEW.Account_No;
    END IF;
END //

DELIMITER ;

-- TEST:
INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount)
VALUES (10004, 'WITHDRAW', 1000);

-- CHECK:
SELECT * FROM Account WHERE Account_No = 10004;

```

Output:

The screenshot shows a database management tool interface. The top bar indicates 100% zoom and a time of 48:23. The 'Result Grid' tab is active, displaying a table with columns: Account_No, Customer_ID, Account_Type, Balance, and Branch. The first row shows data for Account_No 10004, Customer_ID 104, Account_Type Savings, Balance 44000.00, and Branch Chennai. Below this, there are rows with 'NULL' values. To the right of the table is a 'Result Grid' icon and a 'Filter Rows' search bar. Below the table, there are 'Apply' and 'Revert' buttons. The 'Action Output' tab is also visible, showing a log of actions. The first action is a SELECT statement that returned 1 row(s) in 0.00040 seconds.

Account_No	Customer_ID	Account_Type	Balance	Branch
10004	104	Savings	44000.00	Chennai
NULL	NULL	NULL	NULL	NULL

Time	Action	Response	Duration / Fetch Time
12 14:09:48	SELECT * FRO...	1 row(s) returned	0.00040 sec / 0.000...

Question 7 — Trigger — prevent withdrawal when balance is insufficient

Query:

```

DELIMITER //

CREATE TRIGGER PreventInsufficientWithdrawal
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    DECLARE CurrentBalance DECIMAL(12,2);

    SELECT Balance INTO CurrentBalance
    FROM Account
    WHERE Account_No = NEW.Account_No;

    IF NEW.Transaction_Type = 'WITHDRAW'
    AND NEW.Amount > CurrentBalance THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Withdrawal failed: Insufficient balance';
    END IF;
END //

```

```

DELIMITER ;

-- TEST (should fail):
INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount)
VALUES (10005, 'WITHDRAW', 5000000);

```

Output:

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```

30
31 DELIMITER //
32
33 CREATE TRIGGER PreventInsufficientWithdrawal
34 BEFORE INSERT ON Bank_Transaction
35 FOR EACH ROW
36 BEGIN
37     DECLARE CurrentBalance DECIMAL(12,2);
38     SELECT Balance INTO CurrentBalance
39     FROM Account
40     WHERE Account_No = NEW.Account_No;
41
42     IF NEW.Transaction_Type = 'WITHDRAW'
43     AND NEW.Amount > CurrentBalance THEN
44         SIGNAL SQLSTATE '45000'
45         SET MESSAGE_TEXT = 'Withdrawal failed: Insufficient balance';
46     END IF;
47 END //
48
49 DELIMITER ;
50
51
52 -- TEST (should fail):
53 INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount)
54 VALUES (10005, 'WITHDRAW', 5000000);
55

```

The Action Output pane at the bottom shows the following error:

	Time	Action	Response	Duration / Fetch Time
7	10:41:45	-- TEST (shoul...	Error Code: 1644. Withdrawal failed: Insufficient balan...	0.011 sec

Question 8 — Procedure — transfer money between two accounts (with checks)

Query:

```

DELIMITER //

CREATE PROCEDURE TransferMoneyFull(
    IN p_Sender INT,
    IN p_Receiver INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN

```

```

DECLARE v_SenderExists INT DEFAULT 0;
DECLARE v_ReceiverExists INT DEFAULT 0;
DECLARE v_SenderBalance DECIMAL(12,2);

SELECT COUNT(*) INTO v_SenderExists FROM Account WHERE Account_No = p_Sender;
SELECT COUNT(*) INTO v_ReceiverExists FROM Account WHERE Account_No = p_Receiver;

IF v_SenderExists = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Sender account does not exist';
ELSEIF v_ReceiverExists = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Receiver account does not exist';
ELSE
    SELECT Balance INTO v_SenderBalance FROM Account WHERE Account_No = p_Sender;

    IF v_SenderBalance < p_Amount THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient balance for
transfer';
    ELSE
        UPDATE Account SET Balance = Balance - p_Amount WHERE Account_No =
p_Sender;
        UPDATE Account SET Balance = Balance + p_Amount WHERE Account_No =
p_Receiver;
    END IF;
END IF;
END //

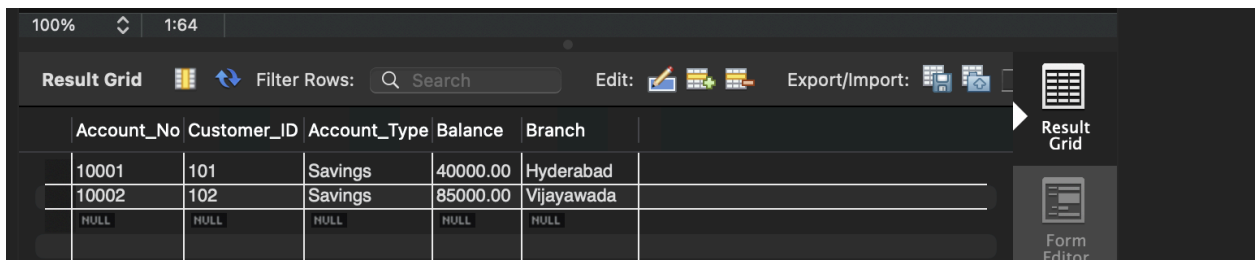
DELIMITER ;

-- EXECUTION:
CALL TransferMoneyFull(10001, 10002, 5000);

-- CHECK:
SELECT * FROM Account WHERE Account_No IN (10001, 10002);

```

Output:



The screenshot shows a database application interface. At the top, there's a toolbar with icons for zooming, filtering, and editing. Below the toolbar is a 'Result Grid' tab. The grid displays the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	40000.00	Hyderabad
10002	102	Savings	85000.00	Vijayawada
NULL	NULL	NULL	NULL	NULL

On the right side of the grid, there are buttons for 'Result Grid' and 'Form Editor'.

Question 9 — Procedure — calculate loan simple interest

Query:

```

DELIMITER //

CREATE PROCEDURE CalculateLoanInterest(
    IN p_LoanAmount DECIMAL(12,2),
    IN p_Rate DECIMAL(5,2),
    IN p_Years INT,

```

```

        OUT p_SimpleInterest DECIMAL(12,2),
        OUT p_TotalAmount DECIMAL(12,2)
    )
BEGIN
    SET p_SimpleInterest = (p_LoanAmount * p_Rate * p_Years) / 100;
    SET p_TotalAmount = p_LoanAmount + p_SimpleInterest;
END //

DELIMITER ;

-- EXECUTION:
CALL CalculateLoanInterest(500000, 7.5, 5, @SI, @Total);
SELECT @SI AS Simple_Interest, @Total AS Total_Amount;

```

Output:

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```

CREATE PROCEDURE CalculateLoanInterest(
    IN p_LoanAmount DECIMAL(12,2),
    IN p_Rate DECIMAL(5,2),
    IN p_Years INT,
    OUT p_SimpleInterest DECIMAL(12,2),
    OUT p_TotalAmount DECIMAL(12,2)
)
BEGIN
    SET p_SimpleInterest = (p_LoanAmount * p_Rate * p_Years) / 100;
    SET p_TotalAmount = p_LoanAmount + p_SimpleInterest;
END //

DELIMITER ;

```

The output is displayed in the Result Grid at the bottom:

Simple_Interest	Total_Amount
187500.00	687500.00

Question 10 — Trigger — log balance changes into Account_Audit

Query:

```

CREATE TABLE Account_Audit (
    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
    Account_No INT,
    Old_Balance DECIMAL(12,2),
    New_Balance DECIMAL(12,2),
    Changed_Date DATETIME DEFAULT CURRENT_TIMESTAMP
)

```

```

);

DELIMITER //

CREATE TRIGGER LogBalanceChange
AFTER UPDATE ON Account
FOR EACH ROW
BEGIN
    IF OLD.Balance <> NEW.Balance THEN
        INSERT INTO Account_Audit (Account_No, Old_Balance, New_Balance)
        VALUES (NEW.Account_No, OLD.Balance, NEW.Balance);
    END IF;
END //

DELIMITER ;

-- TEST:
UPDATE Account SET Balance = Balance + 1000 WHERE Account_No = 10001;

-- CHECK:
SELECT * FROM Account_Audit;

```

Output:

The screenshot shows the MySQL Workbench interface. The SQL editor displays the following SQL code:

```

1 CREATE TABLE Account_Audit (
2     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
3     Account_No INT,
4     Old_Balance DECIMAL(12,2),
5     New_Balance DECIMAL(12,2),
6     Changed_Date DATETIME DEFAULT CURRENT_TIMESTAMP
7 );
8
9 DELIMITER //
10
11 CREATE TRIGGER LogBalanceChange
12 AFTER UPDATE ON Account
13 FOR EACH ROW
14 BEGIN

```

The bottom of the window shows the 'Result Grid' with the following data:

Audit_ID	Account_No	Old_Balance	New_Balance	Changed_Date
1	10001	40000.00	41000.00	2026-08-22 10:50:05
NULL	NULL	NULL	NULL	NULL